PHASE 1 – 4 · INDUSTRIAL DEFECT DETECTION

NEU-DET Industrial Surface Defect Detection

Classification, transfer learning, RetinaNet and YOLOv8 object detection, and Grad-CAM interpretability on the Northeastern University surface-defect benchmark.

6 defect classes | **1,800** labelled images | **4** YOLO experiments | **Grad-CAM** + saliency

TEAM MEMBERS

Judy Hazem · Abdallah Galal · Youssef Sherif

DATASET

NEU-DET (Northeastern University)

FRAMEWORKS

TensorFlow · Keras · Keras-CV · Ultralytics

HARDWARE

NVIDIA RTX 5080 Laptop · CUDA 12.8

1 Abstract

SUMMARY OF THE WORK

This report presents a complete computer-vision pipeline for automatic detection of six classes of surface defects on hot-rolled steel strips, using the NEU-DET benchmark. The work spans four technical phases: dataset preparation and exploratory analysis, baseline convolutional classification, transfer learning with *EfficientNetB0*, and object detection with *RetinaNet* and four *YOLOv8* variants. A baseline CNN trained from scratch reached **94.72 %** validation accuracy, and EfficientNetB0 transfer learning improved this to **97.22 %**. For localisation, the best YOLOv8s configuration at 800-pixel input reached **mAP@0.50 = 0.757** across the 216-image held-out test set, while a YOLOv8-P2 architecture with CLAHE preprocessing and weak-class oversampling delivered the strongest result on the most difficult class, raising the `crazing` AP_{50} from 0.340 (baseline) to **0.528**. Grad-CAM and saliency maps confirm that the classifier attends to the actual defect regions rather than the background.

Keywords: surface defect detection · NEU-DET · convolutional neural networks · transfer learning · EfficientNetB0 · RetinaNet · YOLOv8 · mAP · Grad-CAM · interpretability

2 Introduction & Problem Statement

Surface defects on hot-rolled steel strips are a long-standing quality-control problem in heavy industry. Cracks, inclusions, patches, pitted surfaces, rolled-in scale and scratches reduce the mechanical reliability of steel products and must be detected before downstream processing. Manual inspection is slow, fatiguing and inconsistent, motivating vision-based automation.

The notebook accompanying this report builds an end-to-end deep-learning pipeline that addresses two coupled tasks on the same dataset:

1. **Defect classification** — given a 200 × 200 px patch, predict which of six defect categories it shows.
2. **Defect localisation** — given the same image, return tight bounding boxes around every defect instance with the correct class label.

The work targets four properties that are critical for real industrial use: high accuracy on visually subtle defects (especially `crazing`), robustness to texture-only backgrounds, reproducibility, and explanations that show *where* the model is looking. The notebook explicitly aligns its outputs with four required project phases, summarised in Table 1.

Required phase	Notebook coverage
Phase 1 — Dataset & Baseline CNN	Extraction, cleaning checks, preprocessing, EDA, baseline CNN, accuracy/loss curves, confusion matrix.
Phase 2 — Transfer Learning & Improvement	EfficientNetB0 transfer learning, augmentation, fine-tuning, quantitative comparison against the baseline CNN.
Phase 3 — Object Detection	RetinaNet baseline, YOLOv8m baseline, three improved YOLO experiments, IoU/precision/recall/mAP, 20-image detection demo.
Phase 4 — Model Interpretability	Grad-CAM and saliency maps for ten held-out images plus a discussion of attention quality.

Table 1. Notebook coverage against the four required project phases.

Report structure

3 Dataset Description

The project uses the **NEU Surface Defect Detection (NEU-DET)** benchmark released by Northeastern University, supplied as a single archive (`archive (20).zip`) that the notebook extracts into a `NEU_DET/` working directory. The dataset captures six industrially relevant defect categories that appear on the surface of hot-rolled steel strips: `crazing`, `inclusion`, `patches`, `pitted_surface`, `rolled-in_scale` and `scratches`. Each image is a grayscale-rendered RGB patch of exactly **200 × 200 pixels**, as confirmed by the notebook's image-size audit which reported a single unique `(width, height) = (200, 200)` combination across the full corpus.

3.1 Splits and label files

The notebook treats the archive as already containing a fixed train / validation partition. Bounding-box annotations follow the **PASCAL VOC** XML convention, with one XML file per image describing the defect category and pixel-coordinate `xmin, ymin, xmax, ymax` boundaries. Counts after extraction are summarised in Table 2.

Asset	Training	Validation	Total
Images (`.jpg`)	1,440	360	1,800
VOC XML annotations	1,439	361	1,800
Image–XML pairs successfully matched	1,439	360	1,799
Missing XML files	1 (`crazing_240.jpg`)	0	1

Table 2. NEU-DET image and annotation counts after extraction.

> The notebook explicitly logs the single missing annotation for `crazing_240.jpg`; this image is excluded from the detection-only pipelines but remains usable for classification.

3.2 Class balance

The official split is perfectly balanced at the image level: every class contains exactly **240 training** and **60 validation** images (Fig. 1). This removes per-class image-frequency bias from the classification task. The detection task, however, is *not* instance-balanced: a single image can contain multiple bounding boxes, and the per-class bounding-box totals differ substantially, as analysed in Section 5.

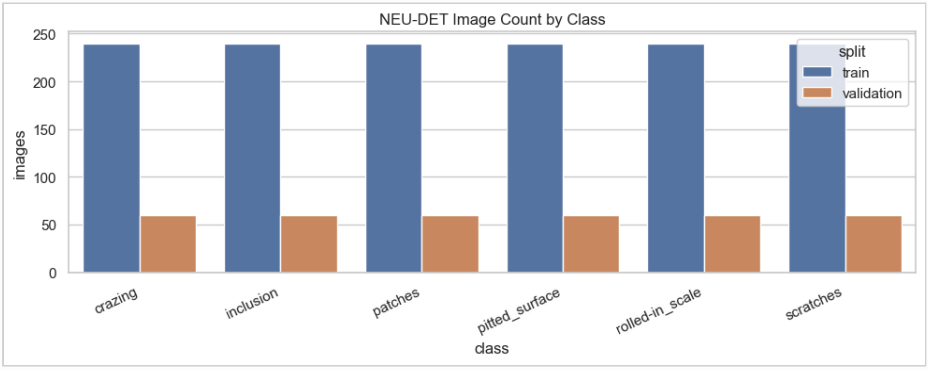

Figure 1. Image counts by class and split, as reported by the notebook. All six categories contribute 240 training and 60 validation images.

3.3 Detection split

For object detection, the notebook builds a third, held-out **test split** by performing a class-stratified 85 / 15 carve-out of the training images. This guarantees representative class coverage in both the reduced training pool and the new test pool, while leaving the original validation folder untouched for model selection (Table 3).

Split	crazing	inclusion	patches	pitted_surface	rolled-in_scale	scratches	Total
train	203	204	204	204	204	204	**1,223**
validation	60	60	60	60	60	60	**360**
test (held-out)	36	36	36	36	36	36	**216**

Table 3. Detection-task partition obtained by class-stratified resampling (seed = 42, test ratio = 0.15).

The **216-image test split** is treated as the canonical evaluation set for every object-detection experiment in the report, so all reported precision, recall and mAP numbers refer to the same images. Image–XML pairs are converted to YOLO-format label files (one `.txt` per image, normalised `cx cy w h`) in a later step.

4 Data Preprocessing & Augmentation

The notebook applies four distinct preprocessing pipelines, each tuned for the model it feeds: a simple rescale for the baseline CNN, an EfficientNet-aware pipeline for transfer learning, a Keras-CV pipeline for RetinaNet, and a YOLO-format conversion (optionally with CLAHE contrast enhancement) for the Ultralytics experiments.

4.1 Baseline classification pipeline

For the baseline CNN, images are loaded with `ImageDataGenerator`, resized to **224 × 224 px** and rescaled by `1/255`. No geometric or photometric augmentation is applied at this stage; the goal is to establish an honest lower bound that the transfer-learning pipeline can be measured against.

4.2 Transfer-learning augmentation

For EfficientNetB0, the training generator uses the ImageNet-trained `efficientnet_preprocess` function in place of the simple rescale, combined with a conservative augmentation policy chosen so as not to destroy the fine textures that distinguish defect classes: rotations up to ±15°, horizontal/vertical shifts and zoom up to 10–15 %, horizontal flipping, and nearest-pixel fill on out-of-frame regions. The validation generator only applies `efficientnet_preprocess` so reported scores reflect un-augmented performance.

4.3 RetinaNet detection pipeline

RetinaNet samples are built by reading each VOC pair, resizing the image to **256 × 256 px**, and rescaling the bounding-box coordinates by the same ratio so that the `xyxy` boxes stay in the new pixel space. Each sample is yielded as a dictionary `{images, bounding_boxes:{boxes, classes}}` consumed by Keras-CV's `RetinaNet` head.

4.4 YOLO-format conversion and augmentation

For the Ultralytics models, every image and XML pair is converted into the YOLO directory layout — `images/{train,val,test}` and `labels/{train,val,test}` — together with a `data.yaml` manifest. Two YOLO datasets are produced: **neu_det_original** (direct VOC→YOLO conversion of the 1,799 annotated images) and **neu_det_clahe_weak** (CLAHE contrast enhancement plus oversampling of the two visually weakest classes, `crazing` and `rolled-in_scale`, in the training split only). The effect of oversampling on box counts is shown in Figure 2 and Table 4: the training pool for `crazing` doubles from 440 to 880 boxes and `rolled-in_scale` roughly doubles from 416 to 832, while the other classes are unchanged.

Figure 2. Training-set bounding-box counts per class for the original YOLO dataset versus the CLAHE + weak-class oversampling variant.

Class	Original	CLAHE + Weak	Change
crazing	440	880	+100 %
rolled-in_scale	416	832	+100 %
inclusion	724	725	±0 %
patches	596	600	±0 %
scratches	361	361	±0 %
pitted_surface	299	299	±0 %

Table 4. Training-set box counts before and after weak-class oversampling.

On top of the dataset-level changes, the YOLO training runs use a shared **BASE_AUGMENTATION** recipe: `mosaic=1.0` (disabled in the last 20 epochs via `close_mosaic=20`), `mixup=0.10`, `copy_paste=0.10`, rotations up to 5°, translation 10 %, scale 0.50, horizontal flip 0.5, vertical flip 0.20, and mild HSV jitter. The baseline YOLOv8m at 512 px uses a softer policy matching the original notebook configuration.

5 Exploratory Data Analysis

Before any modelling, the notebook produces a detailed audit of bounding boxes and visual samples. This step is critical for NEU-DET because the six defect classes differ enormously in how big, how thin and how spatially scattered their instances are, and that geometry directly drives architectural choices for the detector — particularly input resolution and the use of an additional high-resolution P2 detection head.

5.1 Bounding-box geometry

Aggregating all **4,186 bounding boxes** across the corpus and grouping them by class produces Table 5. Two observations dominate. First, `inclusion` is the most frequent class (1,011 boxes) and also the *thinnest*, with a mean box width of only 28.5 px on a 200 px image. Second, `pitted_surface` produces fewer but much larger regions (mean 127 × 173 px). These two extremes flank a middle group (`crazing`, `patches`, `rolled-in_scale`, `scratches`) with medium-scale defects.

Class	N boxes	μ width	σ width	μ height	μ area (px²)
crazing	686	125.0	40.5	75.4	9,233
inclusion	1,011	28.5	14.6	85.1	≈2.4 k
patches	881	56.2	20.7	83.4	≈4.7 k
pitted_surface	432	127.3	52.9	173.0	≈22 k
rolled-in_scale	628	73.1	28.0	77.8	≈5.7 k
scratches	548	61.3	67.1	115.0	≈7.1 k

Table 5. Per-class bounding-box statistics (pixels). Computed directly from the VOC XML files.

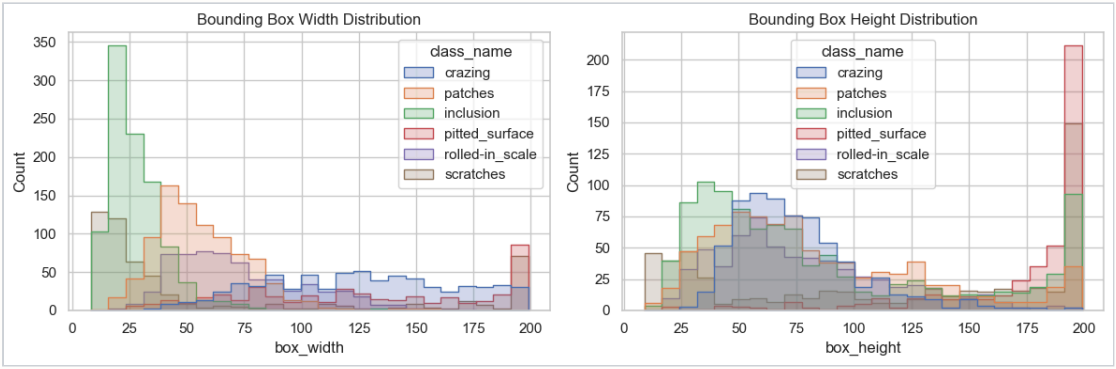

Figure 3. Distribution of bounding-box width (left) and height (right) per class. `inclusion` dominates the small-width regime; `pitted_surface` sits at the upper tail of both dimensions.

5.2 Visual sample per class

Figure 4 overlays the VOC annotations on one representative training image per class. `crazing` is a global, low-contrast texture filling almost the entire frame; `scratches` are very thin diagonal streaks; `inclusion` appears as several small dark spots; and `pitted_surface` covers large regions with weak contrast. These characteristics directly motivate the higher-resolution and weak-class oversampling experiments in Section 8.

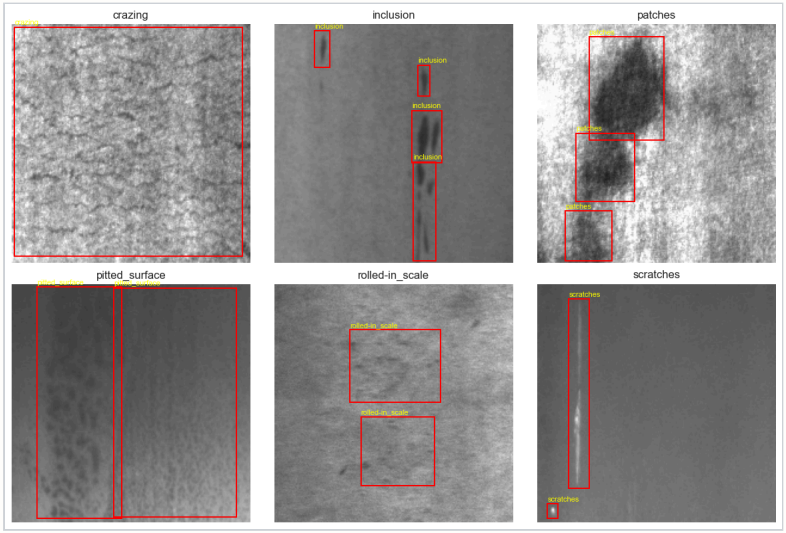

Figure 4. Annotated VOC samples — one representative image per class with ground-truth bounding boxes drawn in red and class labels in yellow.

6 Model Methodology — Classification

The classification stack contains two models trained on the same balanced six-class problem and evaluated on the same 360-image validation set: a convolutional network designed from scratch (the *baseline*) and a transfer-learning model built on top of an ImageNet-pretrained *EfficientNetB0* backbone.

6.1 Baseline CNN

The baseline CNN is a deliberately simple feed-forward stack of four convolutional blocks (32, 64, 128, 256 filters, each followed by 2 × 2 max-pooling), a 256-unit dense head and a 50 % dropout layer before the six-way softmax output. With the 224 × 224 input it has **9,827,398 trainable parameters** (37.5 MB).

Baseline CNN — layer summary

Input	224 × 224 × 3
Conv2D 32 + MaxPool	→ 111 × 111 × 32
Conv2D 64 + MaxPool	→ 54 × 54 × 64
Conv2D 128 + MaxPool	→ 26 × 26 × 128
Conv2D 256 + MaxPool	→ 12 × 12 × 256
Flatten → Dense 256 + ReLU	
Dropout 0.5	
Dense 6 + Softmax	→ class probabilities
Total parameters	**9.83 M**

All weights are initialised from scratch; no pretrained features are used. The optimiser is Adam at `lr = 1 × 10⁻⁴` with categorical cross-entropy loss.

6.2 EfficientNetB0 transfer learning

The transfer-learning model wraps an `EfficientNetB0(weights="imagenet", include_top=False)` backbone. The feature map at the final `top_conv` layer is reduced by global average pooling, passed through a batch-normalisation and a 35 % dropout layer, and finally fed to a six-way softmax head. Two training stages are used:

EfficientNetB0 — layer summary

Input	224 × 224 × 3
EfficientNetB0 backbone (frozen)	→ 7 × 7 × 1280
GlobalAveragePooling2D	→ 1280
BatchNorm + Dropout 0.35	
Dense 6 + Softmax	→ class probabilities
Total params	**4.06 M**
Stage 1 trainable	10,246 (head only)
Stage 2 trainable	last 30 backbone layers + head

Stage 1 trains only the new head for 25 epochs using Adam at 1×10^{-3}. Stage 2 unfreezes the last 30 backbone layers and fine-tunes for a further 15 epochs at the much smaller learning rate 1×10^{-5}, preserving the pretrained low-level filters while letting the high-level features adapt to steel-defect textures.

6.3 Why these two architectures

The pairing follows the standard academic recipe of *from-scratch baseline → pretrained improvement*. The from-scratch CNN benchmarks how much can be learned from 1,440 training images alone; EfficientNetB0 measures the benefit of ImageNet priors plus mild augmentation. Importantly, even though EfficientNetB0 nominally has fewer parameters (4.06 M vs 9.83 M), its *effective representation* is much richer because of the pretraining, which is exactly the property the experiment is designed to test.

7 Training Configuration & Classification Results

7.1 Training configuration

Setting	Baseline CNN	EfficientNetB0 (Stage 1 / Stage 2)
Input resolution	224 × 224	224 × 224
Batch size	32	32
Optimiser	Adam	Adam
Learning rate	1×10^{-4}	1×10^{-3} / 1×10^{-5}
Loss function	Categorical cross-entropy	Categorical cross-entropy
Max epochs	30	25 + 15
EarlyStopping (val_loss)	patience 8, restore best	patience 8, restore best
ReduceLROnPlateau	factor 0.3, patience 3	factor 0.3, patience 3
Augmentation	rescale only	±15°, shift 10 %, zoom 15 %, h-flip

Table 6. Training hyperparameters and callbacks used in the classification stage.

7.2 Learning curves

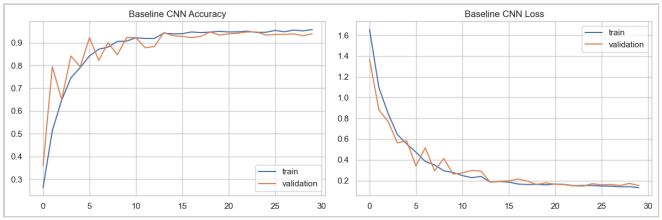

Figure 5. Baseline CNN: training/validation accuracy (left) and loss (right) over the full 30 epochs.

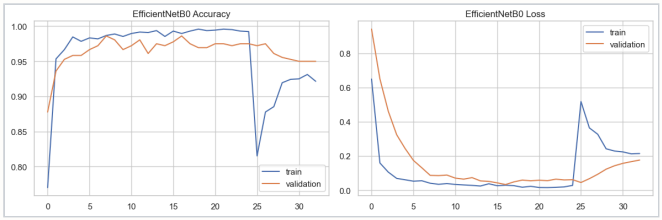

Figure 6. EfficientNetB0 combined curves (Stage 1 followed by Stage 2 fine-tune).

7.3 Validation performance

Both models are evaluated on the 360-image validation set using overall accuracy and the per-class precision / recall / F1 from `sklearn.classification_report`. EfficientNetB0 improves overall validation accuracy by **+2.50 %** while dividing the validation loss by more than three.

Model	Val. accuracy	Val. loss
Baseline CNN (from scratch)	0.9472	0.1469
EfficientNetB0 transfer learning	**0.9722**	**0.0448**

Table 7. Validation comparison for the two classifiers.

Class	Baseline CNN	EfficientNetB0	Δ
crazing	0.99	1.00	+0.01
inclusion	0.92	0.91	−0.01
patches	0.98	1.00	+0.02
pitted_surface	0.87	1.00	+0.13
rolled-in_scale	1.00	1.00	±0.00
scratches	0.92	0.92	±0.00

Table 8. Per-class F1-score on the validation split (60 images per class).

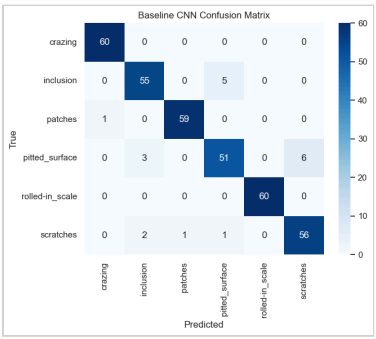

Figure 7. Confusion matrix — baseline CNN.

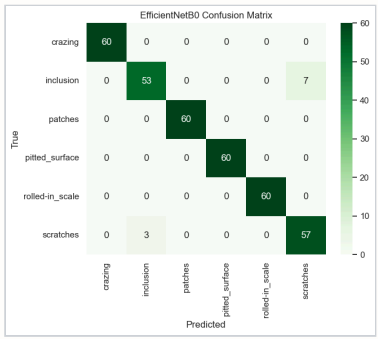

Figure 8. Confusion matrix — EfficientNetB0.

8 Object Detection — RetinaNet & YOLO

8.1 RetinaNet baseline (Keras-CV)

The first detector is the standard *RetinaNet* head on top of a `ResNet50` ImageNet backbone, taken directly from Keras-CV. Inputs are 256 × 256 px, the bounding-box format is `xyxy`, and the loss is the canonical *focal + Smooth-L1* combination.

STAGE 1 — FROZEN BACKBONE

- Backbone `resnet50_imagenet`, trainable = `False`
- Optimiser Adam, `lr = 2 × 10⁻⁵`
- EarlyStopping patience 6 on `val_loss`
- Max 30 epochs, checkpoint on best val_loss

STAGE 2 — PARTIAL FINE-TUNE

- Last 25 backbone layers unfrozen
- Optimiser Adam, `lr = 5 × 10⁻⁶`
- EarlyStopping patience 3, 8 epochs maximum
- Recompiled with the same focal + Smooth-L1 losses

Evaluation on the 216-image test split uses a confidence threshold of **0.25** and an IoU match threshold of **0.50**; for each ground-truth box the notebook records the highest-IoU prediction of the same class. Aggregated results are reported in Table 9.

Mean IoU	Median IoU	$IoU_{0.5}$ match rate	Precision	Recall	TP	FP	FN
0.293	0.174	0.328	0.423	0.328	162	221	332

Table 9. RetinaNet test metrics (216 images, 494 ground-truth boxes).

Per-class $IoU_{0.5}$ match rates make the weakness pattern explicit: `pitted_surface` matches 76 % of ground-truth boxes while `scratches` matches only 7.6 % and `rolled-in_scale` matches 21 %. These are precisely the thin / low-contrast classes identified during EDA.

Class	N GT boxes	Mean IoU	$IoU_{0.5}$ match rate
crazing	83	0.293	22.9 %
inclusion	127	0.285	36.2 %
patches	92	0.388	43.5 %
pitted_surface	46	0.591	76.1 %
rolled-in_scale	80	0.226	21.3 %
scratches	66	0.053	7.6 %

Table 10. RetinaNet per-class $IoU_{0.5}$ match rate on the test set.

8.2 YOLOv8 experiments

RetinaNet's overall match rate of ~33 % at IoU 0.50 is clearly insufficient for industrial use, so the notebook moves to the *Ultralytics YOLOv8* family and runs **four** experiments designed to isolate the effect of architecture size, input resolution, augmentation strength, and dataset preprocessing (Table 11). All four runs are evaluated on the same 216-image test split with the official YOLO validator at `conf = 0.001`, so the precision–recall curve is never clipped.

Experiment	Model	Dataset	imgsz	Epochs	Batch	Optimiser
baseline_yolov8m_512	yolov8m.pt	original	512	120	8	AdamW, lr 0.0008
yolov8s_img800_aug	yolov8s.pt	original	800	200	8	AdamW, lr 0.0008
yolov8m_img800_aug	yolov8m.pt	original	800	180	8	AdamW, lr 0.0006
yolov8_p2_img960_clahe_weak	yolov8-p2.yaml + yolov8s.pt	clahe_weak	960	220	4	AdamW, lr cos schedule

Table 11. YOLOv8 experiment matrix.

All four runs share a cosine learning-rate schedule with `lrf = 0.01`, weight decay `5 × 10⁻⁴`, an EarlyStopping-style `patience` of 25–50 epochs and the augmented policies from Section 4. The fourth experiment additionally enables the **YOLOv8-P2** architecture, which adds an extra high-resolution detection head specifically for very small objects, and pairs it with the CLAHE-preprocessed dataset whose `crazing` and `rolled-in_scale` images are oversampled.

9 YOLO Comparison, Detection Outputs & Interpretability

9.1 Overall YOLO comparison

Table 12 reports the official Ultralytics validator output on the 216-image test split. The baseline YOLOv8m at 512 px already reaches **mAP@0.50 = 0.735**, which is markedly higher than the RetinaNet match rate. Increasing input resolution to 800 px and adopting stronger augmentation lifts mAP@0.50 to **0.757** (best overall) for YOLOv8s and equips the P2 model with the largest gain on the visually weakest class.

Experiment	Precision	Recall	mAP@0.50	mAP@0.50:0.95	crazing AP_{50}	rolled-in_scale AP_{50}
yolov8s_img800_aug	0.748	0.708	**0.757**	0.406	0.501	0.643
yolov8m_img800_aug	0.728	0.702	0.754	0.396	0.451	0.633
yolov8_p2_img960_clahe_weak	0.725	0.696	0.751	0.359	**0.528**	0.633
baseline_yolov8m_512	0.673	0.706	0.735	0.402	0.340	0.623

Table 12. Overall YOLO test metrics, sorted by mAP@0.50. The two rows in colour mark the best overall and best `crazing` experiments.

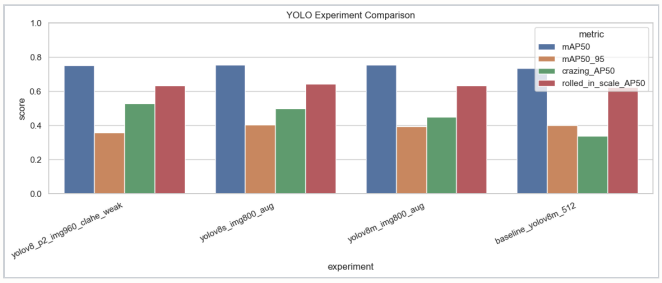

Figure 9. Headline YOLO metrics (mAP@0.50, mAP@0.50:0.95, `crazing` AP_{50}, `rolled-in_scale` AP_{50}) across the four experiments.

Figure 10. Per-class AP_{50} across all four experiments. The CLAHE + weak-class variant clearly improves `crazing` without hurting the other classes.

9.2 Best-model selection and practical evaluation

Using two different selection rules, the notebook designates **yolov8s_img800_aug** as the best *overall* model (mAP@0.50 = 0.757) and **yolov8_p2_img960_clahe_weak** as the best *crazing-focused* model (crazing AP_{50} = 0.528, +0.188 over the baseline). The P2 model is re-evaluated at a realistic `conf = 0.15` and IoU 0.50, returning **precision = 0.730, recall = 0.710, mAP@0.50 = 0.687** — the configuration used for the 20-image qualitative demonstration in Figure 11.

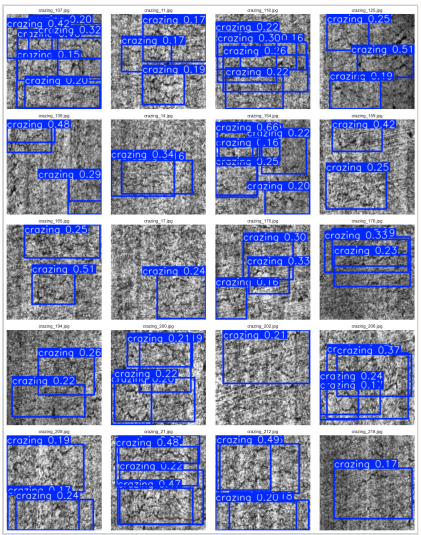

Figure 11. Twenty held-out test images with bounding-box predictions from the selected YOLOv8-P2 model at `conf = 0.15`. Most defects are correctly localised; the hardest cases involve thin, low-contrast scratches and crazing.

9.3 Interpretability — Grad-CAM and saliency

Phase 4 of the notebook attaches a Grad-CAM hook to the last convolutional layer (`top_conv`) of the EfficientNetB0 backbone and additionally computes pixel-level *saliency* maps from the gradient of the predicted class score with respect to the input. Ten class-balanced test images were processed; the predicted class matched the ground truth in **9 of 10** cases, with the only failure being a `crazing` image misclassified as `pitted_surface` at 0.81 confidence — exactly the failure mode anticipated in Section 5 for low-contrast, full-field textures.

10 Discussion, Conclusion & Future Work

10.1 Discussion

Three results from the notebook deserve special emphasis.

Transfer learning is decisively useful. Even with a balanced dataset of 1,440 training images, the from-scratch CNN tops out at 94.72 % validation accuracy whereas EfficientNetB0 reaches 97.22 % with fewer trainable parameters. The improvement is concentrated almost entirely on `pitted_surface`, whose F1 jumps from 0.87 to 1.00, while `inclusion` and `scratches` remain the residual bottleneck for both models.

Resolution and a high-resolution detection head matter for thin defects. Across the YOLO experiments, the largest single source of improvement is moving from 512 px to 800 px input. The crazing-targeted YOLOv8-P2 + CLAHE + oversampling combination then adds another +0.077 to `crazing` AP_{50} over the best 800-px run. The price is a small drop in overall mAP@0.50:0.95 (0.359 versus 0.406), illustrating the practical trade-off between rare-class recall and overall regression quality.

RetinaNet underperforms the YOLO family on this dataset. At its best the RetinaNet baseline matches only 33 % of ground-truth boxes at IoU 0.50, with extremely weak recall on `scratches` (7.6 %). In contrast, every YOLO experiment reports test mAP@0.50 of at least 0.73, suggesting that the modern one-stage YOLO design — with its mosaic augmentation, anchor-free head and built-in multi-scale features — is a better default for this benchmark than the configuration of RetinaNet trained here.

Grad-CAM confirms that the classifier learned defect-specific cues rather than steel-strip background. In nine of ten interpretability images the Grad-CAM activations cluster on the actual defect region; the single misclassified `crazing` example also coincides with a diffuse heatmap that spans most of the image, consistent with the dataset-level observation that `crazing` is a global texture rather than a localised object.

10.2 Limitations

- One annotation is missing from the corpus (`crazing_240.jpg`), so this image is excluded from detection training.
- All experiments were run on a single laptop GPU (RTX 5080 Laptop, 16 GB); the YOLOv8-P2 run was constrained to batch size 4 at 960 px.
- The RetinaNet baseline uses 256 × 256 inputs and a fixed two-stage schedule — it was not tuned as aggressively as the YOLO runs and the gap should not be interpreted as a definitive architectural verdict.
- Per-class instance counts are inherently imbalanced at the box level (inclusion ≈ 1,011 vs pitted_surface ≈ 432) even though image counts are balanced; CLAHE + weak-class oversampling only partially compensates.

10.3 Conclusion

This project delivered a reproducible end-to-end pipeline on the NEU-DET benchmark covering all four required phases. EfficientNetB0 transfer learning improves classification accuracy from 94.72 % to **97.22 %** with both fewer trainable parameters and a 3.3× lower validation loss. For localisation, a YOLOv8s model at 800 px reaches **mAP@0.50 = 0.757** on the held-out 216-image test set, and a YOLOv8-P2 model with CLAHE preprocessing and weak-class oversampling raises `crazing` AP_{50} from 0.340 to **0.528**, addressing the single most difficult class identified in the exploratory analysis. Grad-CAM and saliency visualisations corroborate that the network attends to true defect regions, supporting deployment in inspection workflows where explainability is required.

10.4 Future work

- Add an explicit ensemble of `yolov8s_img800_aug` and `yolov8_p2_img960_clahe_weak` at inference time, since the two best models are strong on disjoint subsets of the class space.
- Retrain RetinaNet at 640 × 640 with focal-loss tuning to obtain a fair two-stage versus one-stage comparison.
- Introduce semi-supervised pseudo-labelling on additional unannotated steel-strip images to expand the training corpus for `scratches` and `crazing`.
- Move beyond Grad-CAM with score-CAM or integrated gradients to cross-validate the interpretability findings.